REVON: LOG AND MERKLE DIFFERENCING FOR DURABLE STRUCTURED DATA VERSIONING

BY ATHARVA SHEERSH PANDEY[1,a], ADHYAN JAIN[1,b], AND POORNIMA NEDUNCHEZHIAN[1,c]

1 School of Computer Science and Engineering, VIT Vellore

a atharva.sheersh2024@vitstudent.ac.in; b adhyan.jain2024@vitstudent.ac.in

c Corresponding author: Poornima Nedunchezhian (poornima.n@vit.ac.in)

Revon is a Python prototype for durable, linear-history key/value versioning. It supports log aggregation and hash-pruned comparison over the same SQLite-backed Merkle hash trie. We evaluate these two diff paths, an operation-count selection rule and the costs of complete versioning workflows. The Linux experiments comprise 516 executions and a separate 72-execution geometry and telemetry supplement. Eight of nine paired hybrid/Merkle timing intervals favour hybrid; the dense case includes parity. A separate threshold control selects Merkle when log is faster. The Windows evaluation covers 93 cases in fresh worker processes, with disjoint calibration and validation histories, controlled sensitivity experiments, Wine Quality and CO2 public sources, and repeated runs at 100,000 and 1,000,000 records. Its calibration winner uses log throughout the tested intervals, leaving the benefit of adaptive switching unresolved. CO2 supplies observed publication changes, whereas wine corrections are generated. We check diff outputs and historical states against semantic oracles and report algorithm, CLI-phase and whole-worker measurements with their respective boundaries. The findings describe trade-offs on the tested machines and histories. They do not establish an optimal crossover, storage-engine superiority or reproduction by an independent team.

1. Introduction. A versioned store for structured data must recover past states and compare them when only part of the data has changed. Saving complete files meets some of this need, but leaves record-level history and comparison to the application. Git provides a familiar model: immutable objects are identified by their content and connected through parent commits [10]. Those objects describe files and directories. A record store must also handle histories in which the same keys persist while update density and the number of intervening commits vary.

We study two ways to compute a diff over a shared durable history. One aggregates operations from a log; the other prunes a Merkle hash trie using equal subtree hashes. Both use the same representation and return the same semantic output. Revon brings together established path-copying, content-addressing and SQLite transaction mechanisms so that we can compare these paths and a rule for selecting between them. The evaluation concerns the measured versioning operations and workloads, rather than the cost of structured-data management as a whole.

Dolt combines SQL, a commit graph and Prolly Trees [13, 28]. ForkBase provides POS-Tree/SIRI storage with fork semantics [32], and Noms describes history-independent chunked collections [3]. Revon focuses on a smaller interface: key/value operations over a linear history. Its purpose here is to support a controlled diff-path comparison, without

Keywords and phrases: data versioning, Merkle hash trie, hybrid differencing, content-addressed storage, benchmark methodology.

claiming feature parity or a new family of persistent indexes. Cross-system measurements use Revon's in-process API and Dolt's CLI. We report the CLI phases separately to show the costs included in that comparison.

1.1. Research Questions. RQ1: What trade-offs in latency, persistent storage and worker resources arise in complete versioning workflows, and how do API/CLI boundaries affect their interpretation?

RQ2: How does an operation-count selector perform on generated histories held out from calibration, compared with forced log and forced Merkle over the same representation?

RQ3: How do workload structure, payload size and fixed-trie geometry affect differencing costs under controlled changes to the tested histories?

1.2. Contributions. Implementation contribution: Revon combines content addressing, path copying and atomic SQLite persistence in a prototype that exposes both diff paths under one encoded-value contract. The engineering contribution is this integration and the ability to compare its paths under matched conditions.

Policy contribution: we define an operation-count rule and compare its choices with both forced paths. Calibration is completed and the selected threshold recorded before held-out evaluation. The analysis includes adverse choices and considers whether always using log is sufficient for the tested histories.

Evidence contribution: separately constructed oracles check the observed diffs and historical states. The reproducibility materials preserve executed sources, raw measurements, environment records and data preparation, together with correctness checks and known telemetry limitations.

Evaluation contribution: the paper reports Linux workflow experiments and a 93-case Windows evaluation using disjoint calibration and validation seeds. The Windows design includes matched sensitivity controls, Wine Quality and CO2 sources, and repeated scale runs. We analyse each campaign according to its own experimental units and do not pool timing rows between campaigns.

The scope is a single-writer store with linear histories and bounded workloads. These experiments can identify trade-offs within that scope. They cannot establish a universally optimal selector or geometry, superiority of a query engine, or reproduction by an independent team.

2. Related Work.

2.1. Content Addressing and Persistent Structures. A Merkle root authenticates the collection represented beneath it [22]. Git uses content addressing for software histories, while IPFS applies it to networked content [10, 5]. Path copying allows persistent structures to retain earlier states without copying everything [14]. Disk-oriented dictionaries include ordered stratified B-trees [31] and fully persistent B-trees [8]. Revon routes hashed keys through a fixed-depth trie: equal hashes can stop a diff traversal, but routing does not provide ordered range access. SciDB and TimeArr address versioned arrays through layouts and deltas suited to that setting [27, 30]. TardisDB supports version access in a main-memory database, and Saquib et al. study replicated versioned structures for wide-area systems [25, 26]. These studies explain several relevant design choices; they do not measure Revon's log and Merkle paths over a common history.

Merkle Search Trees combine search-tree ordering with Merkle authentication to reconcile states [4]. Revon also uses hashes to prune comparisons, but has fixed-depth

routing and a centralized linear history rather than a replicated CRDT. LBFS uses content-defined chunking to find redundant content [23]; Dolt and Noms use related methods to determine chunk boundaries. Such boundaries can preserve deduplication as data shifts, while ordered indexes support range operations. Neither capability is evaluated by the Revon benchmark.

2.2. Structured Dataset Versioning. Dataset versioning involves collaboration as well as the cost of storing and recovering states. DataHub studies collaborative histories at scale [6]. Decibel supports relational branching and compares version-first, tuple-first and hybrid layouts [20]. OrpheusDB partitions data to improve version retrieval when versioning is added to an existing DBMS [19]. Bhattacherjee et al. formalize the storage/reconstruction trade-off [7]. Delta Lake uses transaction logs and checkpoints for ACID tables [2], and version-graph methods select materialized states and deltas under storage and retrieval constraints [18]. The FAIR-compliant proposal concerns consistent release identification and description [16], complementing storage work without specifying a diff algorithm. Revon's key/value interface addresses only part of this broader dataset-management problem.

2.3. System Design Gap. Noms represents structured values in a Merkle DAG and supports diff and synchronization [3]. ForkBase combines content addressing with fork and deduplication semantics [32]. Dolt provides structural sharing, native diff and ordered indexing through content-addressed Prolly Trees [13, 28]. Git4Data exposes snapshot, branch, diff and merge operations through SQL over a relational engine [17]. We use its available arXiv preprint to describe the abstraction, without treating the reported results as peer-reviewed or independently reproduced evidence. Revon has a narrower implementation: fixed-depth hash routing in a single-writer key/value store with linear history, without SQL or a branching interface.

Table 1 summarizes the systems and studies most closely related to these implementation choices.

Table 1
Related-work summary

System or study	Representation or focus	Relevance to Revon
Persistent DS [14]	Path copying	Formal structural-sharing basis
Git [10]	Object DAG	Identity and commit ancestry; file-oriented
IPFS [5]	Merkle DAG	General content-addressed versioning
MST [4]	Merkle search tree	Authenticated ordered reconciliation
DataHub [6]	Version graph	Collaborative dataset motivation
Decibel [20]	Relational layouts	Native database branching
OrpheusDB [19]	Partitioned versions	Storage/retrieval optimization
Delta Lake [2]	Log/checkpoints	ACID time travel at data-lake scale
Noms [3]	Merkle DAG	Structured content addressing
ForkBase [32]	Forkable CAS	Fork and deduplication semantics
DoltHub [13, 28]	Prolly Trees	Mature production comparator

2.4. Design Implications and Research Positioning. Version creation, storage, reconstruction and comparison place work at different points in a history. Complete snapshots make retrieval simple but copy unchanged records. Logs defer work to replay or checkpointing. Path copying shares unchanged structure and creates the paths affected by an update [14, 7]. Decibel and OrpheusDB show how layout affects version creation and retrieval [20, 19]; version-graph analyses make the storage/retrieval trade-off explicit [18]. Reproducibility studies examine how builds and artifacts can be preserved for inspection [1]. We therefore record import, commit, checkout, diff and storage separately, and treat artifact preservation as a condition for inspection rather than evidence of faster execution.

A change can mean more than a difference between stored values. Differential dataflow updates query results as timestamped inputs change [21]. Provenance explains where output facts came from and why they occur [9]. DSLog records fine-grained array lineage and supports queries over it [33]. Explain-Da-V relates semantic dataset changes to transformations [29], while Baseline's operation-based differencing handles refactorings and schema changes [15]. Revon reports keys added, removed or modified between two states. It does not infer causes, lineage, query effects or schema intent. The related studies provide useful context, but their outputs and workloads do not support direct timing comparisons with these key/value diffs.

Each Revon-H commit stores a Merkle root and a content-addressed changeset. Either diff path can therefore recover the same endpoint changes, leaving the selection rule free to choose how that work is done. We hypothesize that log aggregation suits short, sparse intervals and that hash pruning may help after longer operation sequences. Whether their costs cross is an empirical question. Git4Data illustrates the broader interest in database-native version control [17]. Here, Snapshot and Log-only provide representation baselines for a fixed-depth-trie comparison. The public workloads use generated corrections to TLC and wine records, and observed changes in a CO2 publication sequence. These sources do not establish performance across independent hardware, concurrent workloads or production systems.

2.5. Capabilities and comparison boundaries. Table 2 distinguishes the systems' abstractions from the evidence used in this paper. Its capability descriptions are not benchmark results. ForkBase and Noms document typed-collection APIs; we do not evaluate a relational layer over them. Git4Data is represented by a preprint at a different abstraction level [17]. An unevaluated capability should not be read as an unsupported capability.

Table 2
System capabilities and evidence scope; documented features are not performance results

System	Representation / history	Diff / persistence	Branches / queries	Reproducibility and use here
Revon	Fixed-depth hash trie; linear history	Log or Merkle; SQLite objects	No named branches/merge; key/value API	Executed snapshots, semantic proofs and raw trials; measured here
Dolt [13, 28]	Prolly Trees; commit graph	Native version diff; persistent block store	Branches/merge; SQL	Official docs/source; CLI workflow measured, engine not isolated
ForkBase [32]	POS-Tree/SIRI; forkable versions	Hash-pruned diff/merge; addressed chunks	Fork on demand/conflict; typed KV APIs	Primary paper and APIs; no experiment run here
Noms [3]	History-independent chunks; Prolly collections	Hash-based diff/sync/merge; chunk store	Typed collection API; not benchmarked here	Technical overview/source; conceptual comparison
Git4Data [17]	MatrixOne immutable objects/MVCC; table versions	Database-native diff/merge	Snapshot/tag/branch ; SQL extensions	Preprint/BranchBench claims belong to its authors; not run here

2.6. Related foundations and scope boundaries.

Research-data versioning guidance treats version identity and change description as part of reliable citation and reuse [34]. An evaluation of IPFS and the FastVer integrity system address content-addressed persistence and verification in settings broader than Revon's local prototype [35, 36]. Persistent tries, B+-trees, external-memory search trees, snapshot stores, and authenticated data structures offer complementary approaches to retaining and checking prior states [37-41]. Repository views, collaborative analytics, and DEX further illustrate how history systems support repository access and collaborative work [42-44].

Related work on schema evolution and temporal databases studies how data meaning and structure change over time [45-47]. Data-provenance research models the origins and derivations of results, from provenance semirings and scientific-workflow surveys to broader tutorials and syntheses [48-51]. Further work considers provenance for aggregate queries, authenticated or secure provenance, FAIR data stewardship, privacy, and the PROV model [52-56]. Platforms such as SEEK show how provenance and data management are integrated in a scientific repository [57].

The storage literature provides context for the tree and log mechanisms considered here: foundational B-tree work and cache-oblivious indexing explain alternative search and update costs [58-61], while LSM trees, Bigtable, Dynamo, and log-structured file systems represent influential log- or append-oriented storage designs [62-65]. WiscKey and LogBase examine write-efficient key-value and log-based database storage, and multi-version partitioned B-trees address version-aware indexing [66-68]. These systems motivate design comparisons but are not interchangeable with the workloads or measurements evaluated in this paper.

3. Revon Design. Figure 1 shows the implemented architecture and the operations covered by the evaluation.

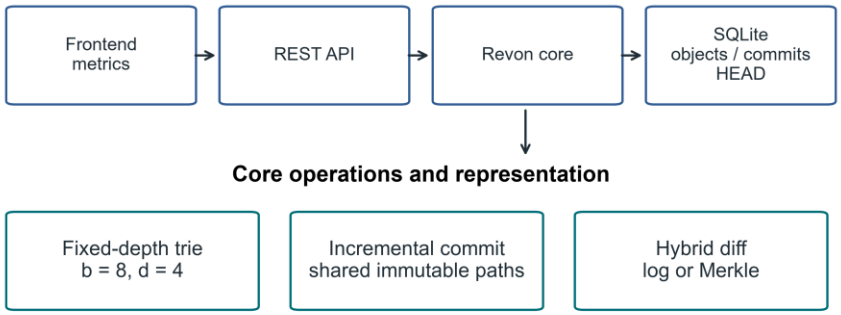


Fig 1. Revon architecture and the implemented research scope

3.1. Addressed Object Model. Revon serializes trie nodes and changesets as canonical JSON and names each object by its SHA-256 digest. A commit's digest covers its root, parent, changeset, message and timestamp. Human-readable labels such as v1 and v2 are aliases, so they do not affect object identity. A durable HEAD reference identifies the current commit.

3.2. Fixed-Depth Merkle Hash Trie. The main configuration uses b=8 and d=4. Routing consumes three bits of SHA-256(key) at each level, selecting among 4,096 possible leaf buckets with twelve bits. Leaves retain full keys, so keys with a shared routing prefix remain distinguishable. Expected occupancy is N/b^d under uniform hashing. We evaluate other b,d combinations separately rather than assuming that this default minimizes cost.

3.3. Incremental Hashing and Commit. For a mutation batch, Revon canonicalizes the values and groups keys that share a routing prefix. It rewrites each affected leaf once, then rebuilds the ancestors along the affected paths. Internal nodes retain unchanged child hashes, allowing the corresponding structure to be shared. A single SQLite transaction writes the objects, changeset, commit metadata, version alias and measurements, and updates HEAD. Reopening a repository verifies stored payload hashes and references.

3.4. Hybrid Diff. Log differencing walks the requested ancestor interval and combines operations by key. Merkle differencing compares roots recursively, stopping at equal hashes and examining entries below unequal paths. Revon-H first validates the version identifiers, checks ancestry in both directions and counts changeset operations. An eligible ancestor interval uses log when the count is no greater than T; otherwise it uses Merkle. Reverse log requests exchange old and new values, and identity requests return an empty diff. Forced log rejects unrelated versions, while unknown identifiers raise an error. Revon-M always requests Merkle, although its public version-pair API also checks ancestry. Both paths return the same sorted semantic diff. Commit and checkout are timed as separate operations.

The Linux workflow matrix uses T=4,096, as fixed by its protocol. A separate Linux control tests T=16,384, chosen by an auxiliary calibration experiment. The threshold counts operations; b^d determines bucket capacity. Neither parameter is fitted to the Linux results.

3.5. JSON value ownership and identity. Revon accepts JSON nulls, booleans, finite numbers, strings, lists and objects with string keys. It validates and detaches input containers before storing them and returns detached containers from read and diagnostic views. Immutable scalars can be shared. Unsupported Python values, nested non-string keys, non-finite numbers and cyclic containers are rejected before a write. Equality follows the v1 JSON encoding: sorted object keys, compact separators, UTF-8 and no NaN. Thus true, 1, 1.0 and signed floating zero retain distinct encodings. This is the prototype's Python encoding contract, not a claim of conformance to a cross-language canonical-JSON standard.

Algorithm 1 (batch update). Validate the batch and take ownership of its values. Before interning any object, reject keys present in both puts and deletes and requests to delete missing keys. Read the old values and retain mutations that change encoded identity. Group them by routing prefix, rewrite each touched leaf, sort its entries and hash its complete payload. Rebuild affected ancestors using unchanged child hashes where possible, then record the content-addressed changeset and commit. Detaching values ensures that callers cannot alter stored history through input containers or supported reads.

Algorithm 2 (log composition). Traverse the ancestor interval in chronological order. For each key, retain its existence and value at the first old endpoint and the last new endpoint. Remove it from the result only when both endpoints agree in existence and encoded value. An insertion followed by deletion cancels, as does an update followed by restoration. A null remains different from an absent key. Sort the remaining keys; for a reverse request, exchange endpoints and swap the added/deleted labels.

Algorithm 3 (selection). Validate the version pair and check ancestry in either direction. Sum changeset operation counts over the interval. Select log for an eligible interval with a count at most T, and Merkle otherwise. Merkle traversal skips equal-hash subtrees and compares values in unequal leaves. Both paths apply the same encoded-identity rule and detach returned values.

Cost model. Let N be the record count, d the depth, b the branching factor, m the keys in a batch, h the interval length, p the accumulated operations, u the distinct interval keys and s the serialized output bytes. Under uniform routing, expected leaf occupancy is N/(b^d); in the worst case it is N. A batch must validate and copy inputs, perform m routed old-value lookups and scan occupied leaves. Touched leaves are sorted and serialized, and affected internal child maps are copied. These costs prevent a dataset-independent O(d) bound for batch processing.

Log composition traverses ancestry and performs O(p + u log u) work, in addition to encoded-value comparisons and output copying/serialization. Merkle cost depends on the node pairs visited, child-slot sorting and leaf entries examined, followed by value comparison and output sorting. Both public version-pair modes traverse ancestry. For a reverse request, the unsuccessful direction may first walk to genesis. Output construction requires work at least proportional to s. Large JSON values add comparison and copying costs that scalar-only benchmarks do not characterize.

Revon loads retained nodes, changesets and commits eagerly from SQLite. Objects are durable on disk but resident while the prototype runs, so the retained history must fit in memory. Opening and verifying a repository also depends on its retained payload. There is no disk-page cache or eviction policy. SQLite transaction atomicity provides a persistence mechanism; production concurrency and operational guarantees require tests beyond this evaluation.

Figure 2 follows selection for a version pair, including eligibility checks and reverse requests.

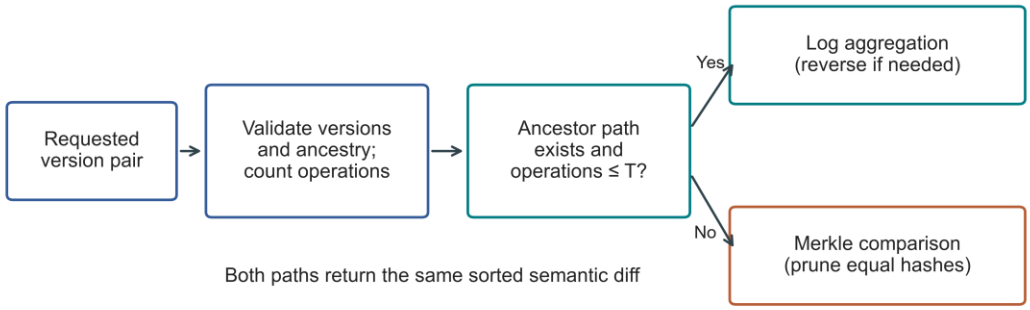


Fig 2. Diff-path selection for a version pair. T is the operation threshold. Invalid versions raise an error before selection, and both paths use the same persistent trie

4. Experimental Methodology.

4.1. Environment and evidence boundaries. The Linux experiments ran on an Ubuntu n2-standard-4 cloud VM. Table 3 records the environment and identifies the executed sources. The Windows experiments ran on a separate computer operated by the authors. Hardware, OS, runtime and storage differ together between the campaigns, preventing attribution of their differences to any one factor. Both campaigns were conducted by the authors.

Table 3
Linux execution environment and source records

Property	Recorded configuration
Machine	GCP n2-standard-4; Intel Cascade Lake; 4 vCPUs; 16 GB RAM; us-east1-c
Operating system	Ubuntu 24.04.5 LTS; x86_64; kernel 7.0.0-1011-gcp
Storage	100 GB balanced persistent disk; no swap configured
Runtime	CPython 3.14.3, Clang 22.1.1 build; GIL enabled; JIT not enabled; SQLite 3.50.4
Comparator and packages	Dolt 2.3.1; psutil 7.1.0; PyArrow 25.0.1
Primary source	linux-validation-20261008: executed source, fixed before measurement
Supplement source	linux-final-supplement-20261008: same core; zero-aware I/O sampler and geometry history checks
Background conditions	Google Ops Agent active; filesystem cache not cleared; monitoring included

Each campaign records its executed source hashes, fixed protocol, environment, correctness checks and raw observations. The Linux preflight suites passed 67 and 69 tests; the Windows suite passed 77. The records include setup failures and the public-data exclusion made before timing. Appendices A and B explain access and give the detailed procedures.

4.2. Variants and workloads. The five systems provide six workflows. Snapshot stores a full state at each version; Log-only stores an initial state and subsequent operations. Revon-M requests Merkle diff, and Revon-H selects a path. Dolt stores records in a primary-key SQL table and returns its native JSON diff. We time SQL INSERT and CSV table import as separate Dolt workflows. Subsequent operations follow the same semantic output contract despite differences in interfaces and storage features. Table 4 defines the workload IDs used in the figures.

Table 4
Linux workloads; all primary histories have ten generated commits

ID / scenario	Rows	Changes / commit	Locality
S1 Small sparse	1,000	10	Spread
S2 Medium sparse	10,000	10	Spread
S3 Medium dense	10,000	1,000	Spread
S4 Large sparse	100,000	100	Spread
S5 Application-key local	100,000	100	Lexically adjacent keys
S6 Range local	100,000	100	Contiguous key intervals
S7 Repeated key	100,000	100	Repeated small key set
S8 Hash-route local	100,000	100	Shared 9-bit hash prefix
P1 TLC public records	100,000	100	Generated fare corrections

The eight synthetic scenarios each use six variants, two warm-ups and seven measured trials, accounting for 432 executions. Public-record workloads contribute 27 executions across Revon-M, Revon-H and Dolt SQL. The threshold controls contribute 54, and the three million-row diagnostics complete the total of 516: 114 warm-ups, 399 measured trials and three diagnostics. Every execution finished with verified output. The accounting does not silently replace or omit failed timing executions.

4.3. Public dataset and generated histories. The TLC workload uses the January 2024 yellow-taxi release [24], whose checksum fixes a Parquet input of 2,964,624 rows. A dispersed affine permutation with seed 20261001 produces nested subsets of 10,000, 100,000 and 1,000,000 rows. We retain eleven fields: vendor, pickup/drop-off times, passenger count, distance, pickup/drop-off locations, payment type, fare, tip and total. Keys combine the release, file-checksum prefix and physical row ordinal. They identify records within that file, without asserting trip identity across releases. Values, including nulls, use compact UTF-8 JSON with sorted keys. The million-row subset contains 47,277 missing passenger counts; every row serializes successfully. Identical field values do not merge rows. This affine sampling procedure is not uniformly random.

We construct ten-commit TLC histories. Each commit changes 100 records in the Linux 100,000-row workload and 1,000 records in the million-row diagnostic. A correction adds 0.01 to fare and total while leaving other retained fields unchanged. All systems receive identical inputs and mutations. These are generated edits, not observed trip revisions. The

public-data builder ignores the configuration's inherited payload_bytes=8 field and retains all eleven source fields; measured serialized bytes determine payload size. Each canonical record is stored as a JSON-encoded string in the common value column. The experiment therefore measures neither native nested-object mutation nor queries on individual fields.

4.4. Timing, correctness and uncertainty. Each workflow proceeds through preparation, repository construction, timed operations, correctness verification and analysis. Import, commit, diff and checkout have separate timing boundaries. A diff returns sorted keys with old/new-value hashes under a common output contract. Table 5 distinguishes algorithm work, interface costs and whole-worker measurements. Correctness checks and decision probes run outside the operation timers.

Table 5
Timing and interpretation boundaries

Quantity	Revon / same-representation control	Dolt CLI workflow	Interpretation
Diff latency	Version API, selected path and canonical output	Native JSON-diff command, parsing and canonical output	Complete interface contract; no engine-only ranking
Phase probe	Separate ancestry/count probe; includes detached-view copying	Command, JSON parse and output-construction timers	Neither probe nor command isolates pure tree-engine work
Correctness	All states, reverse/identity, integrity and reopen outside timers	Observed native rows and historical checkout outside timers	Observed values compared with an independently constructed oracle
Resources	Worker plus sampled children, including oracle/setup	Worker plus sampled Dolt processes	10 ms samples; short processes/peaks may be missed
Persistent bytes	Windows: closed files; Linux: before close/compaction	Repository directory files	Includes metadata/indexes; not isolated algorithm overhead

We compare the observed endpoint diff and every historical state with workload oracles constructed separately from the storage implementation. Revon additionally checks reverse and identity requests, stored-object integrity and reopening. Audits reconstruct the plan and semantic hashes to check internal consistency. They do not repeat the timing experiment.

Linux estimates use seven fresh-repository trials for each fixed history, with paired percentile-bootstrap timing intervals. Windows cases use seven cached queries within a fresh repository and report medians across generated histories or repository repetitions separately. These are different experimental units. Neither campaign samples a population of machines.

The sampler covers the worker and observed descendants, including setup and verification. Its CPU and memory measurements cannot be assigned solely to the storage implementation. Primary Linux read-counter records do not distinguish supported zero values from missing counters. The Linux supplement and Windows sampler make that distinction explicitly. Windows process I/O bytes still do not measure direct physical-disk traffic.

Repository footprint includes persistent metadata and representation costs. Linux directory size is recorded before closing or compacting; Windows size is recorded after closing and the specified verification steps. We keep these measurements separate because their conditioning differs. Neither is a measure of tree overhead alone.

4.5. Frozen threshold and geometry controls. An auxiliary Windows calibration experiment selected T=16,384, the largest candidate in that experiment, without identifying a crossover. The Linux threshold controls use that choice unchanged. The Windows selector experiment below has its own fixed design, executed source and disjoint calibration/validation seeds. Its threshold results are analysed separately.

The Linux control compares forced log, forced Merkle and T=16,384 hybrid over 10,000 rows at 8,192 and 32,768 accumulated operations. Windows outcomes do not determine or retune this control.

The Linux geometry experiment uses five configurations on one fixed spread workload. Windows sensitivity cases use three generated histories and matched equal-capacity geometries. Bucket capacity and the operation threshold are separate parameters. The experiments describe their tested settings rather than establishing a generally optimal geometry.

4.6. Windows calibration and validation. The Windows selector comparison fixes N=12,000, H=8, payload=32 bytes and b=8,d=4. Two calibration seeds and three distinct validation seeds cross spread/repeated-key histories with 64, 512 and 4,096 changes per commit. Each case uses the same durable representation for forced log, forced Merkle and hybrid thresholds T=512/4,096/16,384/65,536. Two warm-ups precede seven measured query blocks, with configuration order deterministically shuffled. Cases start in fresh workers; queries within a case share resident objects and filesystem cache. This is not a cold-cache design.

For calibration, we normalize each hybrid case median by the lower forced-path median, then minimize the equally weighted mean across cases. Exact ties select the smaller T. The threshold and selection time are recorded in UTC before the first validation worker starts; validation does not select a different winner. Sensitivity cases vary rows, history interval, payload, geometry and routing in a prespecified staged design. Holding accumulated operations fixed makes interval length trade off against changes per commit, so the design is not fully factorial. The appendices and protocols specify all seeds and matched controls.

4.7. Public histories and repeated scale experiments. Wine Quality supplies 6,497 red/white records, keyed by colour and source row ordinal [11]. For each of three histories, we generate eight batches of 64 seeded corrections to the quality field. These are not observed specimen revisions. The CO2 package provides published NOAA-derived CSV revisions [12]. Before freezing the experiment, we acquire the eleven most recent file-changing snapshots and exclude one header-only publication failure. Ten populated snapshots yield nine observed transitions, starting from 812 records. Unnamed trailing columns are retained as extra fields. Dates identify records; CSV ordering and formatting do not enter the state comparison. This is an observed repository publication history, not NOAA transaction logs or evidence about atmospheric science.

The three CO2 repositories repeat one observed sequence. Wine and scale repositories instead use distinct generated seeds. Each public source is measured through Revon-M, T=4,096 Revon-H and native Dolt JSON diff, with model order rotated across runs. Scale experiments fix H=4, C=64 and payload=32 at 100,000 and 1,000,000 records, using three seeds per size/system. We report medians and ranges across repository medians descriptively. The accompanying metadata records source URLs, licences, acquisition versions, checksums and transformations. A bounded publication history made source acquisition feasible on the available host without a bulk Software Heritage collection.

Appendix B contains the execution-count matrix in Table 17 and the TLC checksums and Linux public-data protocol in Table 16. The main text defines timing boundaries and

experimental units so that the results can be interpreted without reconstructing the full execution manifest.

5. Linux Results.

5.1. Hybrid versus forced Merkle. Revon-M and Revon-H retain identical persistent structures and changesets, so this ablation changes only the requested diff strategy. Figure 3 and Table 6 include all nine repeated-trial comparisons. Eight hybrid intervals exclude parity; the dense S3 interval contains one. Log access helps on the tested sparse and repeated-key histories. That finding concerns diff execution: selecting log does not reduce the stored representation, and the result does not establish that switching helps every workload.

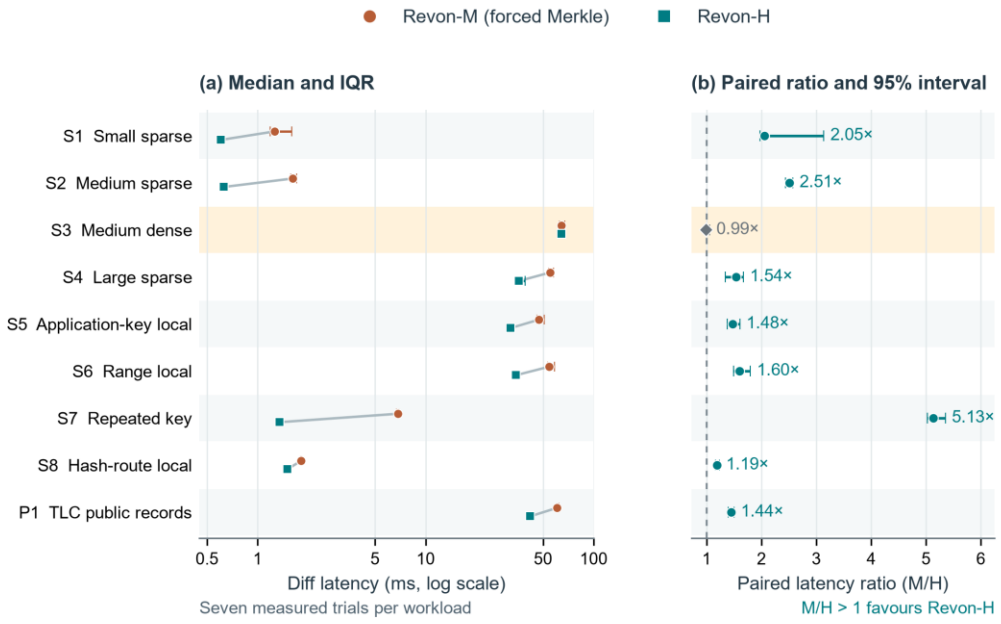


Fig 3. Linux diff comparison for Revon-M and Revon-H. (a) Median latency and interquartile ranges on a logarithmic axis, with workload pairs connected. (b) Median matched-trial M/H ratio and paired percentile-bootstrap 95% intervals from 20,000 resamples. The dashed line marks parity. Each workload contributes seven measured trials. S3 is highlighted because its interval includes parity; the other eight favour H. A median of trial ratios need not equal the ratio of the latency medians. Table 4 defines the workloads. These intervals describe repeated timings of fixed histories on one VM, not variation across independent datasets or machines.

Table 6
Paired M/H diff ablation; milliseconds and median trial ratio [95% interval]

ID	M (ms)	H (ms)	M/H ratio [interval]	H path
S1	1.259	0.604	2.05 [1.97, 3.14]	log
S2	1.620	0.629	2.51 [2.44, 2.57]	log
S3	64.135	64.183	0.99 [0.97, 1.06]	merkle
S4	54.897	35.749	1.54 [1.34, 1.67]	log
S5	47.253	31.967	1.48 [1.38, 1.60]	log
S6	54.352	34.537	1.60 [1.50, 1.80]	log
S7	6.862	1.345	5.13 [5.02, 5.35]	log
S8	1.817	1.503	1.19 [1.17, 1.23]	log
P1	60.597	41.964	1.44 [1.39, 1.50]	log

For P1, median diff latency is 41.964 ms with Revon-H and 60.597 ms with forced Merkle. Both receive the same eleven-field public records and generated edits. S3 contains 10,000 accumulated operations, so H selects Merkle. Its near-parity result is consistent with a shared diff path plus selection work in H.

5.2. Threshold control and adverse result. Figure 4 and Table 7 show both threshold controls, including the adverse choice. Hybrid uses log at 8,192 operations, then switches to Merkle at 32,768. Forced log is faster in all seven matched measured trials at the larger count. An operation count alone therefore does not select the fastest path in these cases. This control tests the fixed choice on Linux; it leaves the best threshold unresolved.

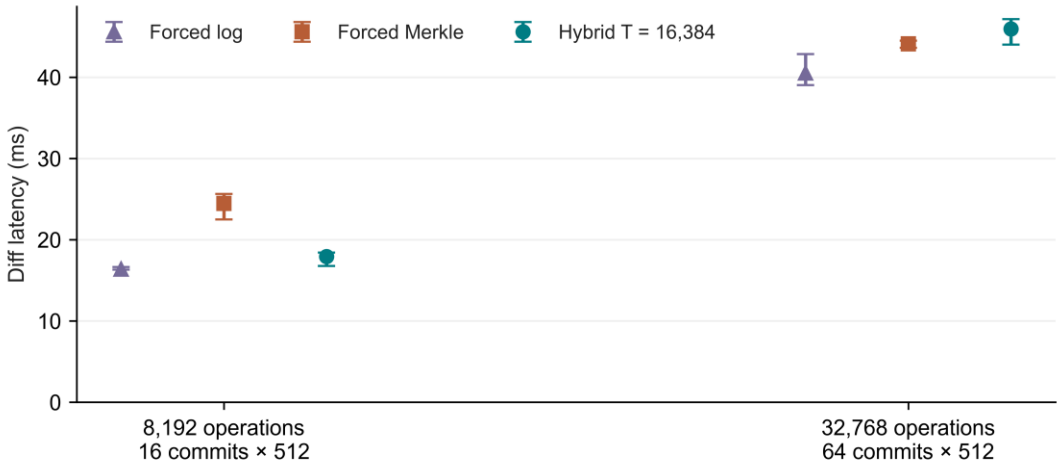


Fig 4. Linux control for the fixed T=16,384 policy. Points and bars show medians and IQRs from seven measured trials per path; labels distinguish the history lengths

Table 7
Frozen threshold controls; diff medians in milliseconds

Operations / history	Forced log	Forced Merkle	Hybrid	Selected path
8,192 / H=16	16.364	24.466	17.926	log
32,768 / H=64	40.520	44.120	45.944	merkle

5.3. Complete workflow comparison. Table 8 compares complete canonical-diff workflows through Revon-H's API and Dolt's SQL/CLI interface. All nine paired Dolt/H intervals lie above one. The timed work includes data structures, process startup, language, serialization, query execution and resident state, so the ratios cannot isolate storage-engine performance. Figure 5 reports commit and checkout separately to show how costs differ across operations.

Table 8
Dolt SQL/CLI versus Revon-H diff; milliseconds and paired ratio [95% interval]

ID	Dolt (ms)	H (ms)	Dolt/H ratio [interval]
S1	90.704	0.604	148.15 [139.77, 160.15]
S2	95.515	0.629	148.70 [140.08, 150.80]
S3	297.617	64.183	4.70 [4.54, 4.77]
S4	152.472	35.749	4.25 [3.84, 4.34]
S5	125.594	31.967	3.73 [3.71, 3.87]
S6	135.765	34.537	3.88 [3.69, 4.12]
S7	97.451	1.345	71.73 [68.95, 74.32]
S8	110.966	1.503	71.24 [68.83, 77.06]
P1	168.576	41.964	4.02 [3.77, 4.07]

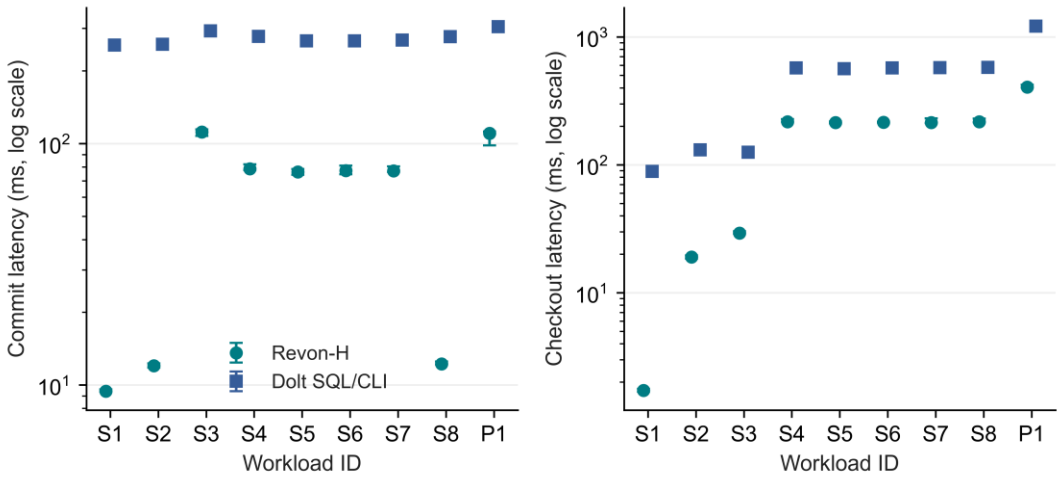


Fig 5. Linux incremental-commit and full-checkout latency for Revon-H and Dolt SQL/CLI. Points are medians and bars are IQRs over seven trials; latency axes are logarithmic

5.4. Storage and memory costs. Figure 6 and Table 9 show persistent directory size and peak RSS for the complete worker alongside the latency findings. Both M and H retain content-addressed changesets and trie data. Their small directory-size difference reflects the measured workflows, not data removed by the log path. Dolt also differs in serialization, compression and indexing. Lower diff latency can therefore accompany a much larger repository; storage and worker resources remain part of the efficiency assessment.

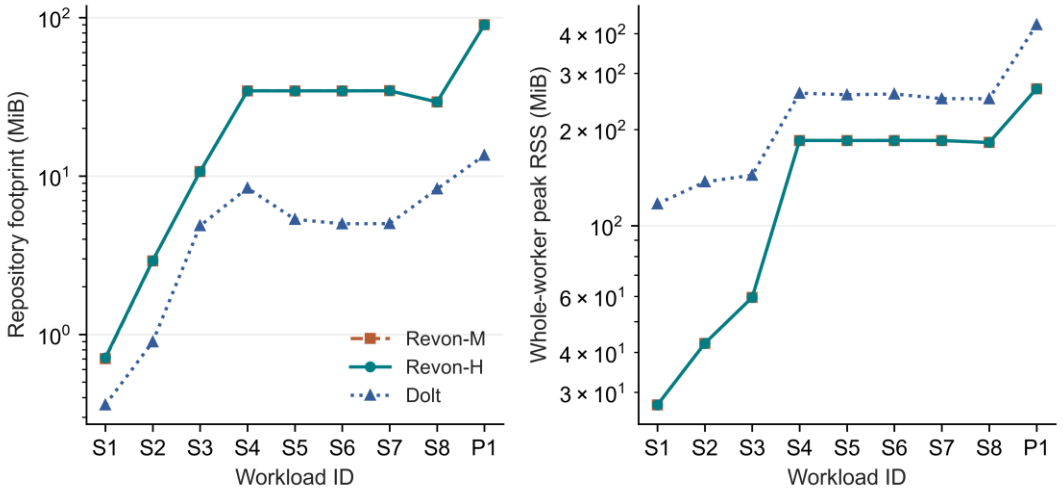


Fig 6. Median repository size before compaction and median whole-worker peak RSS over seven Linux trials, on logarithmic axes. RSS includes shared workload and verification memory

Table 9
Repository footprint and worker RSS; medians in MiB

ID	M disk	H disk	Dolt disk	H RSS	Dolt RSS
S1	0.7	0.7	0.4	27.4	116.9
S2	2.9	2.9	0.9	42.7	137.1
S3	10.7	10.7	4.8	59.5	143.9
S4	34.4	34.4	8.4	184.9	260.1
S5	34.4	34.4	5.3	184.8	257.2
S6	34.4	34.4	5.0	184.9	258.6
S7	34.5	34.5	5.0	184.8	250.0
S8	29.3	29.3	8.3	182.2	249.9
P1	90.1	90.0	13.5	268.6	426.5

5.5. Million-record feasibility. The Linux million-row diagnostic creates and verifies one fresh repository for each of M, H and Dolt. Checks cover all eleven historical states and the common diff oracle. Table 10 gives the individual observations without IQRs, confidence intervals or a ranking. Hybrid uses Merkle because the 10,000 accumulated operations exceed T=4,096. These runs demonstrate completion of the bounded workload on the VM. One observation per system does not establish relative scaling or an optimal configuration. Section 6.4 reports the separate repeated Windows scale experiment.

Table 10
Single-run million-record diagnostics; n=1 per system, no comparative inference

System	Import (s)	Commit (ms)	Diff (ms)	Checkout (s)	RSS (GiB)
Dolt	24.902	547.311	817.034	11.454	2.555
Revon-H	32.289	2961.901	1233.601	4.729	2.561
Revon-M	32.454	2979.717	1265.367	4.685	2.561

5.6. Fixed-trie geometry and telemetry. Figure 7 and Table 11 compare the Linux geometry configurations. The b4-d6, b8-d4 and b16-d3 settings each provide 4,096 leaf buckets. Equal expected occupancy lets us examine routing shape at that capacity. With b=8 fixed, changing depth also changes capacity and the balance between traversal and leaf scanning. Because this experiment uses one fixed workload on one VM, it cannot identify a geometry that is optimal in general.

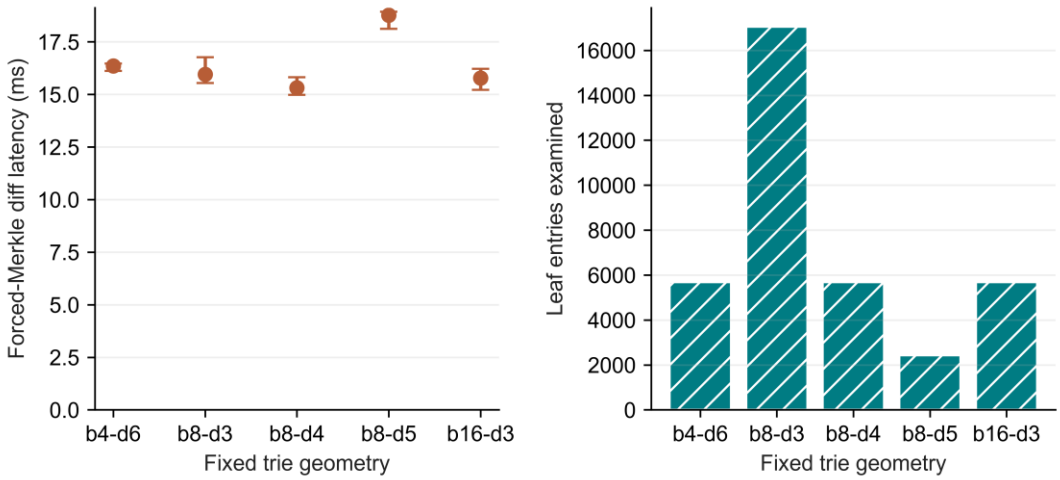


Fig 7. Fixed-trie geometry on Linux. The left panel shows forced-Merkle diff medians and IQRs over seven trials; the right shows median examined leaf entries

Table 11
Geometry work and footprint; one fixed spread workload

Geometry	Buckets	Diff (ms)	Leaf entries	Node pairs	Disk (MiB)
b4-d6	4096	16.355	5682	3628	4.32
b8-d3	512	15.949	17048	585	6.88
b8-d4	4096	15.316	5682	3729	4.25
b8-d5	32768	18.765	2426	6089	6.18
b16-d3	4096	15.775	5682	3929	8.12

Two isolated-worker telemetry tests distinguish valid all-zero I/O samples from unavailable counters, and both pass. The Linux telemetry supplement uses 27 executions on a fixed 10,000-row workload: M, H and Dolt each contribute two warm-ups and seven measured trials. Its records preserve measured zero read bytes instead of blanks. This verifies counter handling, without establishing cold-storage behaviour or ruling out missed samples. The primary latency summaries are analysed separately.

5.7. Representation baselines. Table 12 reports descriptive medians for all six workflows on S4, a representative spread workload. The GitHub reproducibility archive described in Appendix A contains the complete eight-scenario matrix, trial dispersion, normalized storage and separate SQL/CSV import workflows. Snapshot and Log-only serve as representation controls in the common harness, without matching all production features. Their costs of storing complete states or replaying operations provide context for Revon's measurements.

Table 12
Representative S4 workflow medians; complete eight-scenario matrix in the GitHub reproducibility archive

Workflow	Import (s)	Commit (ms)	Diff (ms)	Checkout (ms)	Disk (MiB)
Snapshot	0.071	69.577	624.860	168.158	53.5
Log-only	0.072	2.474	16.483	159.538	5.0
Revon-M	1.488	78.842	54.897	215.661	34.4
Revon-H	1.644	78.590	35.749	217.482	34.4
Dolt SQL	1.663	277.593	152.472	572.585	8.4
Dolt CSV	1.335	273.425	149.325	559.342	7.9

6. Windows Evaluation Results.

6.1. Held-out selector behaviour. Calibration selects T=65,536, the largest tested candidate, with a normalized score of 1.014. Across 18 held-out histories, mean normalized latency is 1.012, compared with 1.101 for T=4,096. The respective agreement counts with the lower forced-path median are 18/18 and 12/18. These describe choices relative to measured, noisy medians; they are not classification accuracy against a known optimum. Table 13 and Figure 8 include every profile group.

The primary Windows intervals contain at most 32,768 operations, so T=65,536 always selects log in this matrix. Its calibration success therefore does not demonstrate a benefit from switching or identify a global crossover. Ratios below one can arise from timing variation even when hybrid and forced execution select equivalent paths. Together with the adverse Linux control, this result limits what can be concluded about the selector. The experimental choice does not change the implementation's default threshold.

Table 13
Held-out median of three history medians; fixed N/H/payload and no population interval

Locality	Operations	Forced log ms	Forced Merkle ms	H4096 ms	Calibrated ms
spread	512	1.913	4.622	1.907	1.837
spread	4,096	25.714	38.878	26.550	27.312
spread	32,768	92.792	104.71	106.56	91.775
repeated-key	512	0.564	1.661	0.549	0.573
repeated-key	4,096	5.608	14.342	6.311	4.664
repeated-key	32,768	62.290	74.366	77.092	67.721

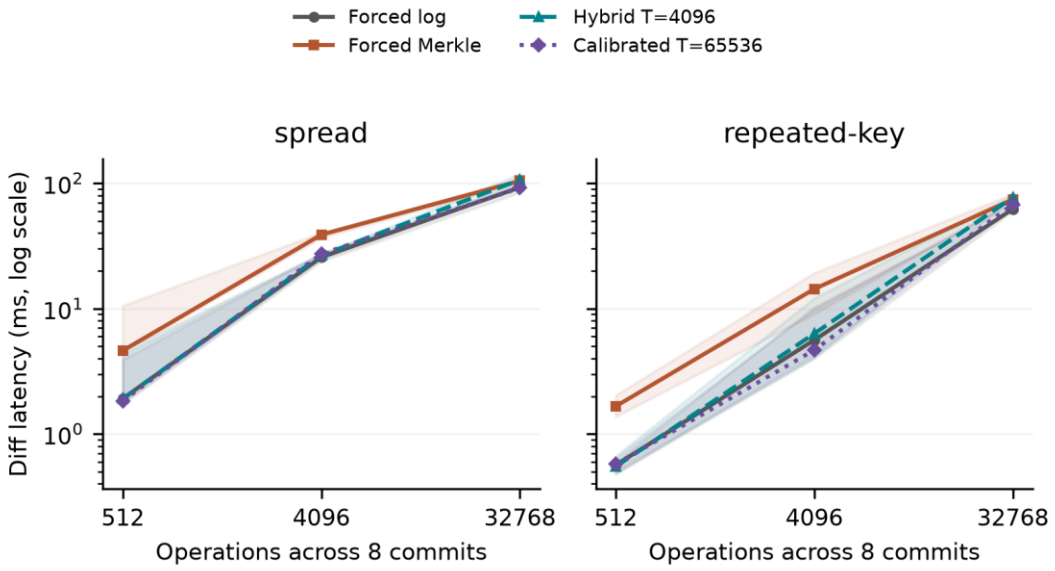


Fig 8. Held-out selector results at N=12,000 and H=8. Lines connect medians of three generated-history medians; shading spans their minimum and maximum. Each history contributes seven cached measured queries. Shading is a descriptive range, not a 95% confidence interval. The calibrated policy selects log at every plotted operation count. All forced paths use the same durable representation.

6.2. Sensitivity and resource interpretation. The decision probe runs outside diff timing and reads a detached public changeset view. Its copying cost therefore prevents interpretation as pure selector overhead. A separate diagnostic extracts the production function prefix covering validation, ancestry and internal operation counting, stopping before dispatch. For the 18 held-out histories and two thresholds, 252,000 measured warm calls give a median of case/threshold medians of 2.674 microseconds, with a range of 1.611-3.678. Its decisions agree with production dispatch. The diagnostic excludes diff traversal and detached-view copying. It is not a cold-cache estimate or a causal correction to subtract from workflow timings. The diagnostic is separate from the 93-case workflow campaign.

The 21 sensitivity cases vary a prespecified factor or matched control across three seeds. They cover payload=256 bytes, N=24,000, fixed-operation H=16, equal-capacity b4-d6/b16-d3 geometries and a C=8 hash-route/spread pair. Geometry cases reproduce the base history's state hashes while changing only routing geometry. Holding operations fixed makes history length trade off against changes per commit, so that comparison does not identify their independent effects. The results describe these profiles rather than selecting a generally best geometry.

Median forced-Merkle latencies were 47.857 ms for more rows, 40.122 ms for longer histories, 39.950 ms for larger payloads, 43.004 ms for b4-d6 and 38.500 ms for b16-d3 geometries, and 0.174 ms for hash-route locality compared with 0.903 ms for its matched spread control. Each value is the median of three history medians. The summaries provide per-history timing, selected paths and resource scopes. These comparisons are exploratory, with no multiple-testing significance claim.

The three sampler off/on pairs produce T=4,096 median-latency ratios from 0.950 to 1.206. Sampling-off runs precede sampling-on runs within each pair; cache state and background activity are uncontrolled. The ratios therefore cannot identify a causal sampling penalty, and we subtract no correction from other timings. Resource counters cover whole

workers. Tests preserve observed zero I/O as a value, but Windows counters cannot identify database logical reads or direct physical-disk traffic.

6.3. Public data and interface phases.

Table 14
Wine and CO2 workflows: median and range of three repository medians; CO2 repeats one observed history

Source / system	Runs	Diff ms	Range ms	Import s	Persistent MiB
co2/revon-m	3	0.612	0.562-0.661	0.033	1.191
co2/revon-h	3	0.309	0.302-0.318	0.035	1.184
co2/dolt	3	256.98	251.87-262.89	0.634	0.178
wine/revon-m	3	4.281	4.260-4.964	0.189	10.832
wine/revon-h	3	2.247	1.956-2.366	0.191	10.812
wine/dolt	3	296.76	246.75-302.81	0.819	1.360

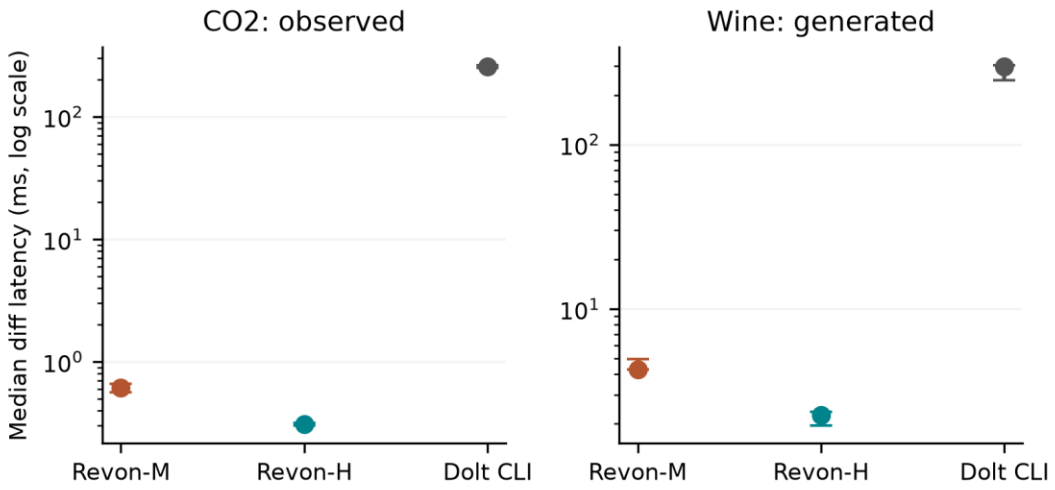


Fig 9. Public-source workflows. Points show medians of three repository medians and bars span their full range. CO2 repositories repeat one observed publication history; wine repositories use three generated correction histories. Each repository contributes seven cached queries. The different API/CLI boundaries make these workflow latencies rather than isolated storage-engine measurements.

For co2-observed, median Dolt command time is 256.063 ms, JSON interpretation 0.501 ms and common output construction 0.503 ms. The command represents 99.6% of median complete diff latency. It includes startup, native work and native JSON emission; this decomposition cannot assign the entire interval to the storage engine or justify subtracting a separately measured startup constant.

For wine-generated, median Dolt command time is 288.464 ms, JSON interpretation 4.445 ms and common output construction 3.258 ms. The command represents 97.2% of median complete diff latency and combines startup, native work and native JSON emission. As with CO2, the measurements do not isolate engine cost or support subtracting an independent startup estimate.

6.4. Repeated Windows scale observations.

Table 15
Repeated Windows scale study: medians of three seed/repository summaries; full ranges in evidence

Rows / system	Import s	Commit ms	Diff ms	Checkout s	Worker RSS GiB
100,000/revon-m	1.116	54.309	5.649	0.209	0.248
100,000/revon-h	1.109	53.044	1.004	0.173	0.242
100,000/dolt	1.662	759.99	283.65	0.662	0.223
1,000,000/revon-m	10.614	88.233	33.694	1.862	1.939
1,000,000/revon-h	10.738	85.362	1.013	1.575	1.925
1,000,000/dolt	11.281	890.97	320.30	3.706	0.976

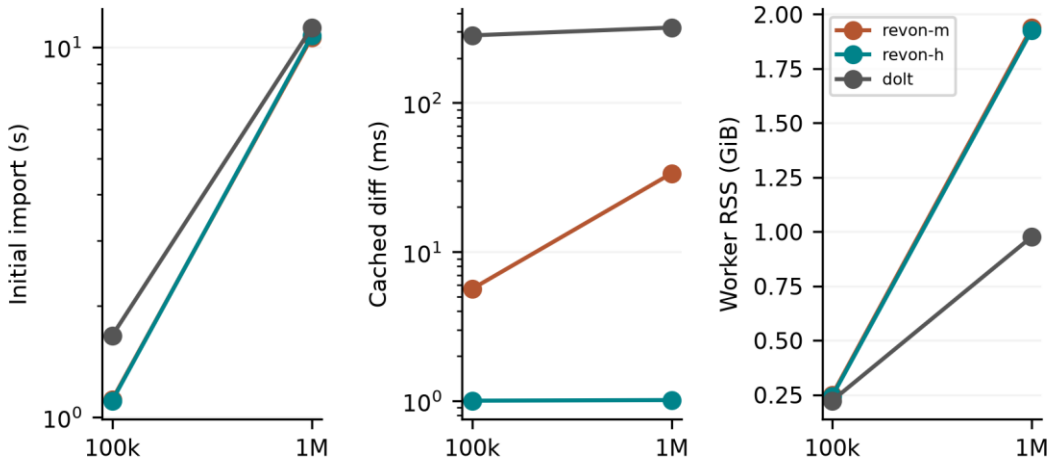


Fig 10. Windows scale observations from three generated histories and fresh repositories per size/system. Lines connect descriptive medians at two sizes; they are not fitted asymptotic laws. Diff queries use cached state, while import and checkout are timed separately. Peak RSS includes workload/oracle generation, storage work, verification and sampled descendants. The Linux million-row n=1 diagnostics use a different campaign.

We analyse the repeated Windows scale runs separately from the Linux diagnostics. Their three histories share an initial synthetic distribution but use different updates; they are not independent public datasets. The manifest records memory availability and background processes, which remain uncontrolled. The measurements describe the tested workflows under those conditions rather than a population ranking or a general scaling law.

6.5. Answers to the research questions. For RQ1, the complete workflows show trade-offs in latency, persistent storage and worker resources, while API/CLI boundaries prevent an engine-only ranking. For RQ2, log helps on several fixed histories, but the Windows calibration winner selects it everywhere tested; an advantage from adaptive switching remains unestablished. For RQ3, controlled histories reveal payload, routing and geometry effects within the staged design without identifying a universal layout. Observed CO2 transitions provide a different kind of history from generated wine corrections and synthetic updates, and each source retains its own scope limits.

7. Discussion. The workflow results address RQ1 at the interface used by an application. Both Linux and Windows experiments measure import, commit, diff, checkout and persistent footprint, but their rows are analysed separately. Snapshot and Log-only expose representation costs; forced M, H and log comparisons hold Revon's representation fixed. Dolt supplies a richer SQL/versioning interface. Reporting its command phases makes the workflow easier to inspect without equating its native tree work to an isolated Python call.

RQ2 requires testing transfer beyond the histories used to select a parameter. The Windows split holds primary N, H, payload and geometry fixed and records selection before validation. Its largest candidate behaves as always-log over the measured range. The adverse Linux threshold choice and dense-case parity show why the rule cannot be assumed to choose the fastest path. A more complex selector would need separate calibration and untouched testing. These findings do not justify changing the default threshold.

For RQ3, geometries with equal capacity can have different traversal depths and ancestor-reconstruction costs despite equal expected occupancy under uniform routing. Payload encoding, touched leaves and output construction also affect latency. Windows cases use separate generated seeds and matched controls, while the Linux geometry experiment holds one history fixed. Neither design estimates every interaction or a population-wide optimum.

Value semantics matter to a valid diff comparison. Revon detaches inputs and outputs so that mutable aliases cannot alter stored history. Its v1 encoded identities distinguish true from 1 and 1 from 1.0, while preserving the object encoding and on-disk schema. Linux trials exercise these semantics on the supplied histories. Unit tests additionally cover null versus absence, nested ownership, unsupported values and rollback.

8. Limitations And Future Work. External validity: the authors ran Linux experiments on one cloud VM and Windows experiments on one local host. These are not independent-party replications or controlled OS comparisons. Hardware, runtime, virtualization and storage change together. Background services and filesystem caching remain active; the Windows manifest also records limited available RAM and process conditions. The study does not represent an entirely idle or cold-cache host.

Workload validity: TLC histories contain generated updates to one monthly release, and Linux scale diagnostics have n=1 per system. Windows validation uses three held-out generated seeds per primary profile, generated wine corrections and one observed CO2 publication sequence. Three CO2 repositories repeat that sequence rather than sampling independent histories. A header-only revision is excluded before timing; unnamed columns are retained without scientific interpretation. Synthetic scale histories share their initial distribution. Broader generalization requires more domains, observed histories and hardware.

Implementation scope: the fixed-geometry trie loads retained nodes, changesets and commits eagerly, without eviction. Revon supports one writer and linear history. Named branches, merge commits, conflict handling, schema evolution, remotes, ordered scans, crash fault injection and concurrent workloads are outside this evaluation. Atomic SQLite transactions alone do not establish all durability or operational guarantees needed for production use.

Measurement and inference: Dolt command time combines process startup, native work and JSON emission. Worker RSS includes oracle and history state, and ten-millisecond sampling can miss brief processes or peaks. Windows I/O counters measure neither physical-disk traffic nor database logical reads. Sampler off/on comparisons also confound

order with cache conditions. Linux bootstrap intervals describe repeated fixed-history timings; Windows three-history ranges are descriptive, without multiple-comparison correction. Source and semantic-hash checks establish internal consistency, not independent timing reproduction.

Selection and design: the primary Windows calibration covers four thresholds, two generated seeds and the specified profiles. The largest candidate, T=65,536, does not switch within the tested range, so broad usefulness of adaptive selection remains unestablished. The staged sensitivity design is not fully factorial: H trades off against density at fixed operation count, and hash-route cases need their matched low-density control. Establishing a different selector or crossover requires a separately fixed experiment.

Possible storage extensions include binary nodes, value chunks, packed objects, compression and reachability-based garbage collection. An ordered Merkle index or content-defined chunking could support a different range-operation design. Each change would require its own semantic and performance evaluation; the fixed-trie results do not predict its effects.

9. Conclusion. Revon provides two semantically consistent diff paths over content-addressed fixed-trie history stored atomically in SQLite. The contribution is an inspectable prototype and an empirical comparison of its workflow and selection costs. Linux evidence covers 516 executions and a separate 72-execution supplement, including dense-case uncertainty and an adverse threshold choice. Windows experiments comprise 93 worker cases and 3,402 checked queries across held-out histories, Wine Quality and CO2 sources, interface phases and repeated scale runs. Calibration selects T=65,536, which always uses log in the tested primary matrix. This outcome leaves an optimal crossover unresolved. The results support conditional trade-offs for the recorded workloads, with API/CLI, sampling, dataset and hardware limits. They establish neither a universal speed advantage nor independent-party reproduction.

Funding: The authors received no funding for this work.

Competing interests: The authors have no competing interests to declare.

Ethics and consent to participate: This computational study involves no recruited participants or human or animal intervention. It uses synthetic histories and publicly released de-identified TLC records. No institutional ethics approval or exemption is claimed.

Consent for publication: The paper includes no identifiable participant material; consent for publication is not applicable.

Data and code availability: The public sources are NYC TLC taxi records [24], UCI Wine Quality [11] and the published CO2 package [12]. Code, executed source snapshots and experimental evidence are available at https://github.com/atharvasheersh/Revon. The paper/ directory contains the Linux reproducibility archive and the Windows reproducibility archive. TLC input records and generated databases are not redistributed. Source metadata documents licences and transformations, and Appendix A explains access.

Acknowledgment. We thank the open-source Dolt project and the researchers whose work on dataset versioning informed the experimental design.

REFERENCES

[1] M. Akhlaghi et al. (2021). Toward long-term and archivable reproducibility, Computing in Science & Engineering, vol. 23, no. 3, pp. 82-91, https://doi.org/10.1109/MCSE.2021.3072860. [Online]. Available: https://arxiv.org/abs/2006.03018.

[2] M. Armbrust et al. (2020). Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores, Proc. VLDB Endowment, vol. 13, no. 12, pp. 3411-3424, https://doi.org/10.14778/3415478.3415560.

[3] Attic Labs. Noms Technical Overview. GitHub. [Online]. Available: https://github.com/attic-labs/noms/blob/master/doc/intro.md.

[4] A. Auvolat and F. Taïani (2019). Merkle Search Trees: Efficient State-Based CRDTs in Open Networks, in Proc. 38th IEEE Int. Symp. Reliable Distributed Syst. (SRDS), pp. 221-230, https://doi.org/10.1109/SRDS47363.2019.00032.

[5] J. Benet (2014). IPFS: Content Addressed, Versioned, P2P File System, arXiv:1407.3561, [Online]. Available: https://arxiv.org/abs/1407.3561.

[6] A. Bhardwaj et al. (2015). DataHub: Collaborative Data Science and Dataset Version Management at Scale, in Proc. 7th Biennial Conf. Innovative Data Syst. Research (CIDR), Open copy: https://www.cidrdb.org/cidr2015/Papers/CIDR15_Paper18.pdf.

[7] S. Bhattacherjee, A. Chavan, S. Huang, A. Deshpande, and A. Parameswaran (2015). Principles of Dataset Versioning: Exploring the Recreation/Storage Tradeoff, Proc. VLDB Endowment, vol. 8, no. 12, pp. 1346-1357, https://doi.org/10.14778/2824032.2824035.

[8] G. S. Brodal, S. Sioutas, K. Tsakalidis, and K. Tsichlas (2020). Fully persistent B-trees, Theoretical Computer Science, vol. 841, pp. 10-26, https://doi.org/10.1016/j.tcs.2020.06.027. [Online]. Available: https://cs.au.dk/~gerth/papers/tcs20.pdf.

[9] P. Buneman, S. Khanna, and W.-C. Tan (2001). Why and where: A characterization of data provenance, in Database Theory: ICDT Lecture Notes in Computer Science, vol. 1973, 2001, pp. 316-330, https://doi.org/10.1007/3-540-44503-X_20. [Online]. Available: https://www.pure.ed.ac.uk/ws/files/16509989/Why_and_Where_A_Characterization_of_Data_Provenance.pdf.

[10] S. Chacon and B. Straub (2014). Git Internals: Git Objects, in Pro Git, 2nd ed., [Online]. Available: https://git-scm.com/book/en/v2/Git-Internals-Git-Objects.

[11] P. Cortez, A. Cerdeira, F. Almeida, T. Matos and J. Reis (2009). Wine Quality [Dataset], UCI Machine Learning Repository, https://doi.org/10.24432/C56S3T. CC BY 4.0.

[12] DataHub Contributors (2026). CO2 PPM - Trends in Atmospheric Carbon Dioxide, NOAA-derived published data package, accessed October https://github.com/datasets/co2-ppm. ODC-PDDL-1.0; acquired revision identifiers and file hashes are retained in the experiment manifest.

[13] DoltHub (2026). Dolt Storage Engine: Block Store, Dolt Documentation, accessed Oct. 2, [Online]. Available: https://www.dolthub.com/docs/architecture/storage-engine/block-store/.

[14] J. R. Driscoll, N. Sarnak, D. D. Sleator, and R. E. Tarjan (1989). Making data structures persistent, J. Comput. Syst. Sci., vol. 38, no. 1, pp. 86-124, https://doi.org/10.1016/0022-0000(89)90034-2.

[15] J. Edwards and T. Petricek (2025). Baseline: Operation-based evolution and versioning of data, arXiv:2512.09762, preprint, updated 2026. [Online]. Available: https://arxiv.org/abs/2512.09762.

[16] A. González-Cebrián, M. Bradford, A. E. Chis, and H. González-Vélez (2024). Standardised versioning of datasets: A FAIR-compliant proposal, Scientific Data, vol. 11, art. 358, https://doi.org/10.1038/s41597-024-03153-y. [Online]. Available: https://www.nature.com/articles/s41597-024-03153-y.

[17] H. Gou, Z. Zhang, Y. Sun, P. Xu, F. Tian, L. Wang, and J. Wang (2026). Git4Data: Database-native version control for AI agents, arXiv:2609.02106, preprint. [Online]. Available: https://arxiv.org/abs/2609.02106.

[18] A. Guo, J. Li, P. Sukprasert, S. Khuller, A. Deshpande, and K. Mukherjee (2024). To store or not to store: A graph theoretical approach for dataset versioning, in Proc. IEEE Int. Parallel and Distributed Processing Symposium (IPDPS), pp. 479-493, https://doi.org/10.1109/IPDPS57955.2024.00049. [Online]. Available: https://arxiv.org/abs/2402.11741.

[19] S. Huang, L. Xu, J. Liu, A. J. Elmore, and A. Parameswaran (2017). OrpheusDB: Bolt-on Versioning for Relational Databases, Proc. VLDB Endowment, vol. 10, no. 10, pp. 1130-1141, https://doi.org/10.14778/3115404.3115417.

[20] M. Maddox et al. (2016). Decibel: The Relational Dataset Branching System, Proc. VLDB Endowment, vol. 9, no. 9, pp. 624-635, https://doi.org/10.14778/2947618.2947619.

[21] F. McSherry, D. G. Murray, R. Isaacs, and M. Isard (2013). Differential dataflow, in Proc. 6th Biennial Conf. Innovative Data Systems Research (CIDR), [Online]. Available: https://www.cidrdb.org/cidr2013/Papers/CIDR13_Paper111.pdf.

[22] R. C. Merkle (1988). A digital signature based on a conventional encryption function, in Advances in Cryptology: CRYPTO '87, Lecture Notes in Computer Science, vol. 293, pp. 369-378, https://doi.org/10.1007/3-540-48184-2_32.

[23] A. Muthitacharoen, B. Chen, and D. Mazières (2001). A low-bandwidth network file system, in Proc. 18th ACM Symp. Operating Syst. Principles (SOSP), pp. 174-187, https://doi.org/10.1145/502034.502052.

[24] New York City Taxi and Limousine Commission (2024). TLC Trip Record Data: Yellow Taxi, January public dataset, accessed Oct. 1, 2026. [Online]. Available: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page.

[25] N. Saquib, C. Krintz, and R. Wolski (2023). Replicated versioned data structures for wide-area distributed systems, IEEE Transactions on Parallel and Distributed Systems, vol. 34, no. 1, pp. 207-224, https://doi.org/10.1109/TPDS.2022.3217969. [Online]. Available: https://racelab.cs.ucsb.edu/papers/tpds22-versioned.pdf.

[26] M. E. Schüle, L. Karnowski, J. Schmeißer, B. Kleiner, A. Kemper, and T. Neumann (2019). Versioning in main-memory database systems: From MusaeusDB to TardisDB, in Proc. 31st Int. Conf. Scientific and Statistical Database Management (SSDBM), pp. 169-180, https://doi.org/10.1145/3335783.3335792. [Online]. Available: https://www-db.cs.tum.edu/~schuele/data/tardis.pdf.

[27] A. Seering, P. Cudré-Mauroux, S. Madden, and M. Stonebraker (2012). Efficient versioning for scientific array databases, in Proc. IEEE 28th Int. Conf. Data Engineering (ICDE), pp. 1013-1024, https://doi.org/10.1109/ICDE.2012.102. [Online]. Available: https://dspace.mit.edu/entities/publication/a5c2c050-6abd-47c1-8bee-4f8c51b6a15f.

[28] T. Sehn (2024). Dolt's Storage Engine, DoltHub Blog, Feb. 29, [Online]. Available: https://www.dolthub.com/blog/2024-02-29-storage-engine/.

[29] R. Shraga and R. J. Miller (2023). Explaining Dataset Changes for Semantic Data Versioning with Explain-Da-V, Proc. VLDB Endow., vol. 16, no. 6, pp. 1587-1600, https://doi.org/10.14778/3583140.3583169.

[30] E. Soroush and M. Balazinska (2013). Time travel in a scientific array database, in Proc. IEEE 29th Int. Conf. Data Engineering (ICDE), pp. 98-109, https://doi.org/10.1109/ICDE.2013.6544817. [Online]. Available: https://scidb.cs.washington.edu/paper/ICDE13_conf_full_422.pdf.

[31] A. Twigg, A. Byde, G. Miłos, T. Moreton, J. Wilkes, and T. Wilkie (2011). Stratified B-trees and versioning dictionaries, in Proc. 3rd USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage), [Online]. Available: https://arxiv.org/abs/1103.4282.

[32] S. Wang et al. (2018). ForkBase: An Efficient Storage Engine for Blockchain and Forkable Applications, Proc. VLDB Endowment, vol. 11, no. 10, pp. 1137-1150, https://doi.org/10.14778/3231751.3231762.

[33] J. Zhao and S. Krishnan (2024). Compression and in-situ query processing for fine-grained array lineage, arXiv:2405.17701, [Online]. Available: https://arxiv.org/abs/2405.17701.

[34] J. Klump, H. Pampel, L. Rothfritz, and D. Strecker (2024). Recommendations on Data Versioning. Zenodo. https://doi.org/10.5281/zenodo.13743875

[35] D. Trautwein, A. Raman, G. Tyson, I. Castro, W. Scott, M. Schubotz, et al. (2022). Design and evaluation of IPFS. Proceedings of the ACM SIGCOMM 2022 Conference, pp. 739-752. https://doi.org/10.1145/3544216.3544232

[36] A. Arasu, B. Chandramouli, J. Gehrke, E. Ghosh, D. Kossmann, J. Protzenko, et al. (2021). FastVer: Making Data Integrity a Commodity. Proceedings of the 2021 International Conference on Management of Data, pp. 89-101. https://doi.org/10.1145/3448016.3457312

[37] E.D. Demaine, S. Langerman, E. Price (2010). Confluently Persistent Tries for Efficient Version Control. Algorithmica, vol. 57, no. 3, pp. 462-483. https://doi.org/10.1007/S00453-008-9274-Z

[38] S. Lanka, E. Mays (1991). Fully persistent B+-trees. ACM SIGMOD Record, vol. 20, no. 2, pp. 426-435. https://doi.org/10.1145/119995.115861

[39] G.S. Brodal, C.M. Rysgaard, R. Svenning (2023). External Memory Fully Persistent Search Trees. Proceedings of the 55th Annual ACM Symposium on Theory of Computing, pp. 1410-1423. https://doi.org/10.1145/3564246.3585140

[40] C. Allain, B. Clément, A. Moine, G. Scherer (2024). Snapshottable Stores. Proceedings of the ACM on Programming Languages, vol. 8, no. ICFP, pp. 338-369. https://doi.org/10.1145/3674637

[41] A. Miller, M. Hicks, J. Katz, E. Shi (2014). Authenticated data structures, generically. Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, pp. 411-423. https://doi.org/10.1145/2535838.2535851

[42] V. Salis, D. Spinellis (2019). RepoFS: File system view of Git repositories. SoftwareX, vol. 9, pp. 288-292. https://doi.org/10.1016/j.softx.2019.03.007

[43] A. Bhardwaj, A. Deshpande, A.J. Elmore, D. Karger, S. Madden, A. Parameswaran, et al. (2015). Collaborative data analytics with DataHub. Proceedings of the VLDB Endowment, vol. 8, no. 12, pp. 1916-1919. https://doi.org/10.14778/2824032.2824100

[44] A. Chavan, A. Deshpande (2017). DEX. Proceedings of the 2017 ACM International Conference on Management of Data, pp. 171-186. https://doi.org/10.1145/3035918.3064056

[45] J.F. Roddick (1995). A survey of schema versioning issues for database systems. Information and Software Technology, vol. 37, no. 7, pp. 383-393. https://doi.org/10.1016/0950-5849(95)91494-K

[46] G. Ozsoyoglu, R. Snodgrass (1995). Temporal and real-time databases: a survey. IEEE Transactions on Knowledge and Data Engineering, vol. 7, no. 4, pp. 513-532. https://doi.org/10.1109/69.404027

[47] C. De Castro, F. Grandi, M.R. Scalas (1997). SCHEMA VERSIONING FOR MULTITEMPORAL RELATIONAL DATABASES. Information Systems, vol. 22, no. 5, pp. 249-290. https://doi.org/10.1016/S0306-4379(97)00017-3

[48] T.J. Green, G. Karvounarakis, V. Tannen (2007). Provenance semirings. Proceedings of the twenty-sixth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems, pp. 31-40. https://doi.org/10.1145/1265530.1265535

[49] Y.L. Simmhan, B. Plale, D. Gannon (2005). A survey of data provenance in e-science. ACM SIGMOD Record, vol. 34, no. 3, pp. 31-36. https://doi.org/10.1145/1084805.1084812

[50] L. Carata, S. Akoush, N. Balakrishnan, T. Bytheway, R. Sohan, M. Seltzer, et al. (2014). A primer on provenance. Communications of the ACM, vol. 57, no. 5, pp. 52-60. https://doi.org/10.1145/2596628

[51] B. Glavic (2021). Data Provenance. Foundations and Trends in Databases, vol. 9, no. 3-4, pp. 209-441. https://doi.org/10.1561/1900000068

[52] Y. Amsterdamer, D. Deutch, V. Tannen (2011). Provenance for aggregate queries. Proceedings of the thirtieth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems, pp. 153-164. https://doi.org/10.1145/1989284.1989302

[53] F. Jamil, A. Khan, A. Anjum, M. Ahmed, F. Jabeen, N. Javaid (2018). Secure provenance using an authenticated data structure approach. Computers & Security, vol. 73, pp. 34-56. https://doi.org/10.1016/j.cose.2017.10.005

[54] M.D. Wilkinson, M. Dumontier, I.J. Aalbersberg, G. Appleton, M. Axton, A. Baak, et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. Scientific Data, vol. 3, no. 1, article 160018. https://doi.org/10.1038/sdata.2016.18

[55] B. Pan, N. Stakhanova, S. Ray (2023). Data Provenance in Security and Privacy. ACM Computing Surveys, vol. 55, no. 14s, pp. 1-35. https://doi.org/10.1145/3593294

[56] L. Moreau, P. Groth, J. Cheney, T. Lebo, S. Miles (2015). The rationale of PROV. Journal of Web Semantics, vol. 35, pp. 235-257. https://doi.org/10.1016/j.websem.2015.04.001

[57] K. Wolstencroft, S. Owen, O. Krebs, Q. Nguyen, N.J. Stanford, M. Golebiewski, et al. (2015). SEEK: a systems biology data and model management platform. BMC Systems Biology, vol. 9, no. 1, article 33. https://doi.org/10.1186/s12918-015-0174-y

[58] D. Comer (1979). Ubiquitous B-Tree. ACM Computing Surveys, vol. 11, no. 2, pp. 121-137. https://doi.org/10.1145/356770.356776

[59] Y.S. Kwong, D. Wood (1980). On B-trees. Proceedings of the 1980 ACM SIGMOD international conference on Management of data - SIGMOD '80, pp. 207. https://doi.org/10.1145/582250.582282

[60] W.J. Hansen (1981). A Cost Model for the Internal Organization of B
⁺
-Tree Nodes. ACM Transactions on Programming Languages and Systems, vol. 3, no. 4, pp. 508-532. https://doi.org/10.1145/357146.357152

[61] M.A. Bender, E.D. Demaine, M. Farach-Colton (2005). Cache-Oblivious B-Trees. SIAM Journal on Computing, vol. 35, no. 2, pp. 341-358. https://doi.org/10.1137/S0097539701389956

[62] P. O'Neil, E. Cheng, D. Gawlick, E. O'Neil (1996). The log-structured merge-tree (LSM-tree). Acta Informatica, vol. 33, no. 4, pp. 351-385. https://doi.org/10.1007/s002360050048

[63] F. Chang, J. Dean, S. Ghemawat, W.C. Hsieh, D.A. Wallach, M. Burrows, et al. (2008). Bigtable. ACM Transactions on Computer Systems, vol. 26, no. 2, pp. 1-26. https://doi.org/10.1145/1365815.1365816

[64] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, et al. (2007). Dynamo. Proceedings of twenty-first ACM SIGOPS symposium on Operating systems principles, pp. 205-220. https://doi.org/10.1145/1294261.1294281

[65] M. Rosenblum, J.K. Ousterhout (1992). The design and implementation of a log-structured file system. ACM Transactions on Computer Systems, vol. 10, no. 1, pp. 26-52. https://doi.org/10.1145/146941.146943

[66] L. Lu, T.S. Pillai, H. Gopalakrishnan, A.C. Arpaci-Dusseau, R.H. Arpaci-Dusseau (2017). WiscKey. ACM Transactions on Storage, vol. 13, no. 1, pp. 1-28. https://doi.org/10.1145/3033273

[67] H.T. Vo, S. Wang, D. Agrawal, G. Chen, B.C. Ooi (2012). LogBase. Proceedings of the VLDB Endowment, vol. 5, no. 10, pp. 1004-1015. https://doi.org/10.14778/2336664.2336673

[68] C. Riegger, I. Petrov (2024). Storage Management with Multi-Version Partitioned BTrees. Information Systems, vol. 125, pp. 102403. https://doi.org/10.1016/j.is.2024.102403

APPENDIX A: REPRODUCIBILITY

The Revon GitHub repository provides the code and experiment records accompanying this paper. Its paper/ directory contains the Linux reproducibility archive and the Windows reproducibility archive. The README files describe dependency setup, public-data acquisition, experiment execution and evidence verification [1]. Each experiment bundle includes its executed sources, fixed protocol, raw measurements and internal audits.

The primary Linux, supplementary Linux and Windows campaigns use distinct protocols and must be analysed separately. TLC input records and generated repositories can be rebuilt with the documented preparation steps. Public-source metadata records licences, acquisition versions, checksums and the CO2 exclusion made before timing. Run auditors on disposable copies with localized data paths to preserve the collected records. Matching semantic hashes checks consistency; it neither reproduces elapsed times nor guarantees identical timing on another machine.

APPENDIX B: METHODS AND EVIDENCE DETAILS

The Linux bundles record executed sources and hashes, environments, fixed workloads, execution order, raw observations and audits. Workers use PYTHONHASHSEED=20261008. Before timing, the primary and supplementary campaigns pass 67 and 69 unit tests respectively. A supplement preflight detects an incomplete transfer that omits the test_data directory. Measurement starts only after restoring that empty fixture directory and passing the complete suite. The setup failure remains in the execution records.

Each primary execution starts in a fresh worker and constructs a repository. Model order is deterministically shuffled and rotated within repetition blocks. Import ends when the first durable version exists. Incremental-commit latency is the median of the individual commit timers for that history. Diff compares the first and last versions and emits sorted changed keys with old/new-value hashes as canonical JSON. Checkout writes the full sorted key/value state as canonical UTF-8 JSON. Dolt timing includes CLI launch, JSON parsing and output normalization; Revon uses an in-process API. The different boundaries prevent interpreting workflow ratios as engine-only speed comparisons.

Correctness checks run outside operation timers. Each primary execution verifies initial and final state hashes, every H+1 historical state and the observed canonical diff against a separately constructed workload oracle. Verification records contain both observed and expected hashes. The audit regenerates workloads, checks matrix membership, selected paths and source/data/protocol identities, and recomputes summaries. Downloaded records

are also audited locally. These checks establish internal integrity rather than an independent repetition of the timing experiment.

After excluding warm-ups, we report medians and interquartile ranges (IQRs) over seven measured repository trials. For latency ratios, matched trials are paired before calculating the median ratio and a paired percentile-bootstrap interval from 20,000 resamples. Values above one favour the denominator. The exploratory 95% intervals concern repeated timings of a fixed workload on one VM, without a population or multiple-comparison interpretation. The median of paired ratios can differ from the ratio of the two latency medians. Every completed trial, including timing outliers, remains in the record.

A parent process samples worker and descendant RSS, CPU and I/O every 10 ms. Peaks include workload generation, oracle state, verification and storage work; they cannot be assigned to the storage engine alone. Brief peaks or short-lived subprocesses can fall between samples. The primary Linux sampler records an all-zero read counter as missing, so those records support no read-I/O conclusion. The supplementary campaign distinguishes measured zero from unavailable counters and is analysed separately. Cached data can be examined logically while the process records zero physical read bytes.

Linux repository footprint is directory size before closing or compaction, including indexes, metadata and representation costs. Records also provide logical-payload normalization and selected first-trial compaction probes. These describe the complete workflow rather than tree overhead alone. CPU, RSS and I/O cover the worker lifecycle, a wider boundary than the individual operation timers.

The auxiliary Windows threshold experiment uses non-overlapping predefined calibration and validation profiles and candidates of 1,024, 2,048, 4,096, 8,192 and 16,384 operations. Minimizing normalized calibration latency selects T=16,384 before validation. Its held-out normalized score is 1.032, compared with 1.297 for T=4,096; always-log scores 1.000 in both splits. These cached-query results belong to that experiment's recorded source and environment and are separate from Linux timings. A largest-candidate winner does not identify an optimum. Long-history and three-seed pilot records provide exploratory context.

The Linux threshold control fixes T=16,384 over 10,000 rows with 512 changes per commit. H=16 and H=64 produce 8,192 and 32,768 accumulated operations. Forced log, forced Merkle and hybrid use separate workers, with two warm-ups and seven measured trials each. The threshold is not selected from Linux outcomes. These cases test transfer of the auxiliary calibration choice, rather than performing another calibration/validation cycle.

The Linux geometry workload has 10,000 rows, ten commits, 100 changes per commit, 32-byte synthetic values and seed 20260824. We force Merkle diff for b4-d6, b8-d3, b8-d4, b8-d5 and b16-d3. Each configuration uses fresh workers for two warm-ups and seven measured trials, with order rotated to limit time/configuration confounding. Every execution checks eleven historical states and the endpoint diff outside the timers. The adapter output contract matches the workflow experiment. Resource peaks from the geometry adapter's in-process monitor are kept separate from external RSS measurements.

The Windows protocol and case manifest specify calibration seeds 202610161/202610162 and separate validation/sensitivity seeds 202610171-202610173. Workers use PYTHONHASHSEED=20261016. Monitoring uses seed 202610190, generated public corrections use 202610201-202610203, and scale uses 202610221-202610223. Queries share a repository within a worker; each worker constructs fresh persistent state. The threshold is timestamped before validation. Selection is not repeated after validation, and Linux/Windows rows are not pooled.

Windows Dolt instrumentation uses the native JSON-diff command and the adapter's parser contract with explicit phase timers. Preflight checks compare its actual output with

the native adapter and the independent oracle. Revon queries check observed output hashes, all historical-state hashes, reverse/identity requests, object integrity and reopening outside timing. Setup checks and data-quality exclusions occur before measurement and remain documented. The retained query matrix and timing-source hashes identify the execution conditions.

The portable audit operates on a disposable evidence copy. In a fresh interpreter it imports the executed source, localizes the recorded public-data path, regenerates workloads and state/diff oracles, checks every configuration, trial and order identity, and recomputes numerical summaries. It then verifies that collected-file hashes are intact. The archives provide these checks and data-acquisition recipes for consistency verification; a timing reproduction requires running the experiments themselves.

Table 16
Public-data checksums and Linux protocol

Item	Definition
Source	NYC TLC yellow_tripdata_2024-01.parquet; 2,964,624 source rows
Raw SHA-256	c4d59da7bbc8abaeeeb1727947ee93d9891a71acb42854bd80db1571b2030510
Prepared million-row SHA-256	6608b7680e82432ca22f90a71d4994676ffd290c4fc85f17f91fab03d4c7b661
P1	100,000 records; H=10; C=100; 2 warm-ups + 7 measured trials per system
Scale diagnostic	1,000,000 records; H=10; C=1,000; one trial per system; no interval or ranking

Table 17
Windows execution matrix; cached queries are distinct from independent history or repository repetitions

Component	Fresh workers	Histories / control	Warm-up queries	Measured queries
Calibration	12	2 seeds × 2 localities × 3 densities	144	504
Held-out	18	3 different seeds × same profiles	216	756
Sensitivity	21	3 seeds × 7 matched variations	252	882
Monitor diagnostic	6	3 off/on pairs, one fixed history	72	252
Public sources	18	2 sources × 3 runs × M/H/Dolt	36	126
Scale	18	2 sizes × 3 seeds × M/H/Dolt	36	126
Total	93	Different campaigns are never pooled	756	2646